

App Dev Project Report

1. Student Details

Name: Talbiya Parveen

Roll Number: 24f2001644

Email: 24f2001644@ds.study.iitm.ac.in

2. Project Details

Project Title: Placement Portal Application

Problem Statement:

To design and develop a web-based Placement Portal Application that provides a centralized platform for managing campus placement activities. The system enables students to view placement opportunities, companies to manage placement drives, and administrators to monitor and control the complete placement process.

Approach:

The application was developed using Flask as the backend framework with a modular architecture. The backend follows a structured approach using models, routes, and service layers for better maintainability.

The system provides role-based access for three types of users:

- **Admin:** Manages students, companies, placement drives, and analytics.
- **Company:** Creates placement drives and manages recruitment activities.
- **Student:** Views available opportunities and applies for placement drives.

The frontend was developed using Vue.js with Bootstrap styling to provide a responsive and user-friendly interface. SQLite is used as the database for storing application data.

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** to assist in writing SQLAlchemy model definitions, creating API documentation samples, and improving variable naming consistency.

The extent of AI/LLM usage is around **10–15%**, limited to **code suggestions and documentation formatting**.

All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Vue.js	Frontend user interface development
Bootstrap 5	Frontend styling and responsive design
SQLite	Lightweight local database for storing user data
Redis	Caching and performance improvement
Celery	Background task processing
Chart.js	Visualization of statistics and analytics
Flask-JWT-Extended	Authentication and authorization using JWT
Flask-Bcrypt	Secure password hashing

5. Database Schema / ER Diagram

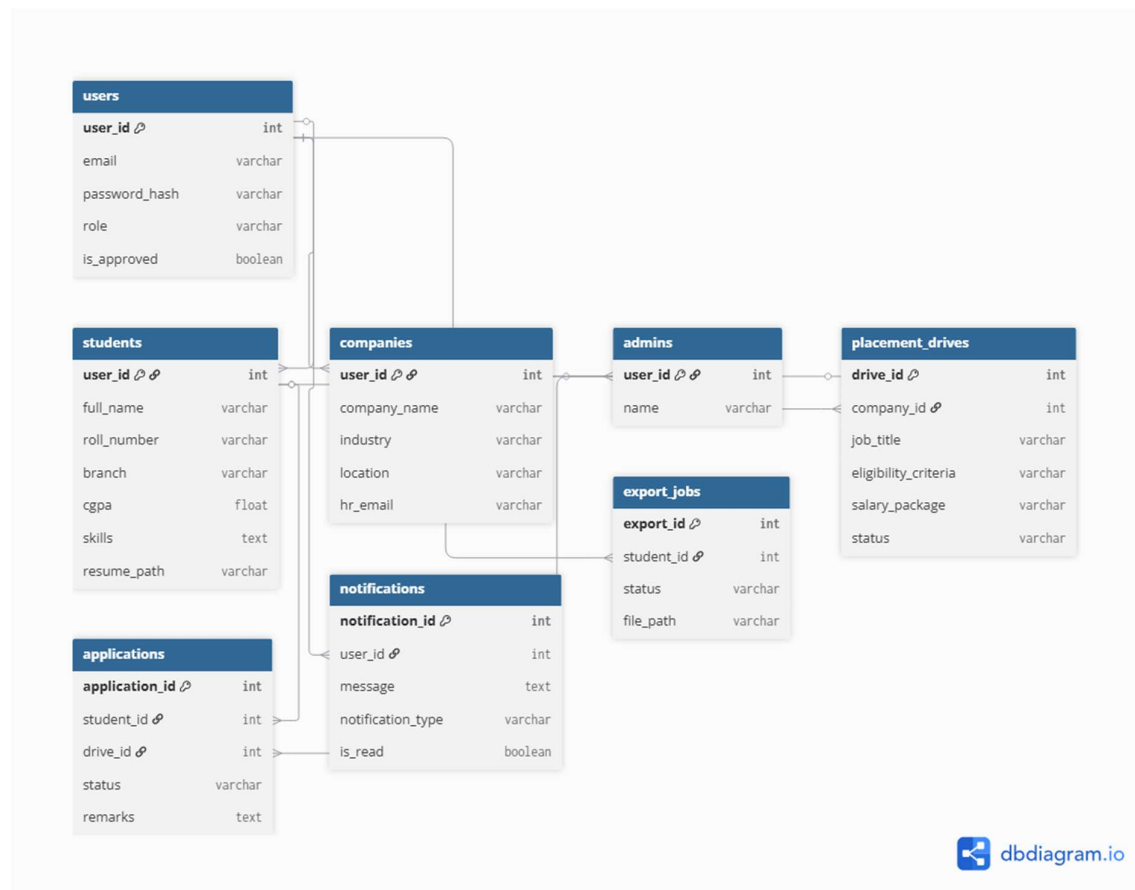
Tables:

1. **User** — stores authentication and user details (user_id, email, password_hash, role, approval status)
2. **Student** — stores student profile details (user_id, name, roll_number, branch, CGPA, skills, resume_path)
3. **Company** — stores company details (user_id, company_name, industry, location, HR details)
4. **PlacementDrive** — stores placement opportunities (drive_id, company_id, job_title, eligibility criteria, salary package, status)
5. **Application** — stores student applications (application_id, student_id, drive_id, application status, remarks)
6. **Admin** — stores administrator details (user_id, name, created_at)

7. **Notification** — stores user notifications (notification_id, user_id, message, notification_type, is_read)
8. **ExportJob** — stores export requests (export_id, student_id, status, file_path)

Relationships:

- One-to-One → User → Student
- One-to-One → User → Company
- One-to-One → User → Admin
- One-to-Many → User → Notification
- One-to-Many → Company → PlacementDrive
- One-to-Many → Student → Application
- One-to-Many → PlacementDrive → Application
- One-to-Many → Student → ExportJob



6. API Resource Endpoints

Endpoint	Method	Description
/api/auth/login	POST	Authenticate user and generate JWT token
/api/auth/register/student	POST	Register a new student account
/api/admin/dashboard	GET	Fetch admin dashboard statistics
/api/company/drives	GET/POST	View and create placement drives
/api/student/drives	GET	Fetch available placement opportunities
/api/student/drives/{drive_id}/apply	POST	Apply for a placement drive

7. Architecture and Features (optional)

Architecture Overview:

The application follows a modular Flask-based architecture.

- app.py – Main Flask application entry point responsible for initializing extensions and registering routes.
- models/ – Contains SQLAlchemy database models for users, students, companies, placement drives, applications, and notifications.
- routes/ – Contains Flask Blueprints for authentication, admin, company, and student APIs.
- services/ – Contains business logic and database operations.
- frontend/ – Vue.js based user interface with responsive design.

Implemented Features:

- User registration and JWT-based authentication
- Role-based access for Admin, Company, and Student
- Student profile management
- Company registration and approval workflow
- Placement drive creation and management
- Student application tracking
- Admin dashboard with placement statistics
- Responsive user interface
- Secure password hashing

Additional Features:

- CSV export functionality for placement records
 - Notification system for users
 - Redis caching for improved performance
 - Background export processing using Celery
 - Search and filtering support for records
-

8. Video Presentation

- **Drive**

Link:

https://drive.google.com/file/d/1vRILfGMyA_qS1x17Kfr2Z7gwD1UtdGQP/view?usp=sharing
